What's New in MiniScript 2.0

MiniScript 2.0 is a complete rewrite of MiniScript, focused on performance and on closing some long-standing gaps in the language. It is still MiniScript: the great majority of 1.x programs run unchanged. This document covers what you may need to fix, and what you can use going forward.

The reference point throughout is **MiniScript 1.6.2**, the last of the 1.x line.

CONTENTS

Why upgrade 2

Changes you may need to make 2

- ^ is now right-associative 2
- super.x = v now stores into self 2
- Assigning an unqualified local from a global is now an error 2
- Strings are indexed in Unicode code points 3
- More consistent numeric formatting 3
- Operators on unsupported types return an error 4

New features 4

- Errors are values 4
- Frozen (immutable) maps and lists 5
- Lists, maps, and computed default parameter values 6
- Property setters 6
- info(x) 7
- function is an expression 8
- Dot syntax on numeric literals 8
- Maps iterate in insertion order 8
- range and [x] * n are now cheap 9
- rnd and shuffle are deterministic across platforms 9
- Mark-and-sweep garbage collector on all platforms 9
- New in command-line MiniScript 9

What hasn't changed 10

Why upgrade

Speed, mostly. Across our benchmark suite the C++ build of 2.0 runs 5-50x faster than 1.6.2 depending on the workload, averaging around 25x; the C# build averages about 3x faster. Beyond that, 2.0 adds real error handling, immutable values, property setters, and a handful of smaller conveniences described below.

Changes you may need to make

`^` is now right-associative

```
print 2^3^2
```

```
512
```

1.x gave `64`, treating this as `(2^3)^2`. Every other language with a power operator — and standard math notation — reads it as `2^(3^2)`. The 1.x behavior was a mistake, and 2.0 corrects it.

> **NOTE**
>
> This only matters for a chain of three or more `^` operands with no parentheses, which is rare in practice.

`super.x = v` now stores into `self`

In 1.x, assigning through `super` wrote into the parent map. In 2.0 it resumes the setter search above the current map and, finding nothing, performs an ordinary store into `self`. Storing into a prototype was never a useful thing to do; the new reading is what makes setters work. If you relied on the old behavior, name the parent map directly.

Assigning an unqualified local from a global is now an error

```
count = 0
bump = function
    count = count + 1
end function
```

```
Compiler Error: illegal assignment to unqualified local 'count' based on nonlocal [line
3]
```

1.x warned about this (since 2022), but then proceeded to do what you almost certainly not want: it read the global, then created a *local* of the same name. 2.0 rejects it at compile time. This error will appear whenever the left-hand side of the assignment is unqualified (does not involve a dot), and is not a local variable already assigned previously in the same function, and the expression on the right-hand side involves the same identifier (explicitly as above, or implicitly like `+= 1`).

To fix the error, explicitly use `globals` or `locals` on either side of the assignment, for example:

```
count = 0
bump = function
    globals.count = count + 1
end function
```

Strings are indexed in Unicode code points

`s[i]`, `len`, slices, `indexOf`, `insert`, `values`, `indexes`, `split("")` and fractional `*` all count code points, on every platform.

```
s = "a" + char(128512) + "z"
print s.len
print s[1]
print s.indexOf("z")
```

```
3
😀
2
```

The 1.x implementations disagreed here: the C++ one counted characters, while the C# one counted UTF-16 code units and so indexed a string containing an emoji differently. 2.0 commits to the C++ reading on both sides.

> **NOTE**
>
> A code point is not always a user-perceived character. A combining accent or an emoji with a skin-tone modifier counts as more than one code point.

More consistent numeric formatting.

MiniScript 1.x was inconsistent in its formatting of numbers (and non-numbers such as NaN and infinity) between the C# and C++ versions. MiniScript 2.0 is consistent, and prints numbers according to these rules:

- `NaN`, `Inf` and `-Inf` are printed exactly as shown here.
- Numbers $> 2 \hat{} 53$ are printed in scientific notation.
- Whole numbers $\leq 2 \hat{} 53$ are printed as plain integers, no decimal point.

- `-0` is printed as just `0`.
- Non-integers with magnitude > 1E10 or < 1E-6 are printed in scientific notation.
- All other non-integers are printed in decimal form with at least one and at most six decimal places (trailing zeros trimmed).

Operators on unsupported types return an error

An operator applied to types it doesn't support now returns an `error` value, where 1.x returned `null`, quietly treated `null` as 0, or sometimes halted.

```
print 1 + null
print [1, 2] + 3
print 1 < "42"
```

```
error: Type error: can't apply '+' to number and null
error: Type error: can't apply '+' to list and number
error: Type error: can't compare number and string
```

In particular:

- `null` in arithmetic is an error on either side: `1 + null`, `null * 3`, and so on. (Joining `null` to a string still works: `"a" + null` is `"a"`.)
- `<`, `<=`, `>` and `>=` work only between two numbers or two strings. (`==` and `!=` still compare any two values.)
- Repeating a string or list a non-finite number of times, as in `"ab" * (1/0)`, or dividing one by 0, is an error.

New features

Errors are values

MiniScript 2 introduces one new data type, `error`. A failure that used to halt your program, or be returned as a string result, now often returns an `error` value you can inspect:

```
lines = file.readLines("/nope/missing.txt")
if lines isa error then print "could not read: " + lines.message
```

```
could not read: File error: readLines: cannot open: /nope/missing.txt
```

Create your own with `err(message)`, optionally wrapping an inner error that represents the cause:

```
cause = err("disk full")
e = err("could not save game", cause)
print e.message
print e.inner.message
```

```
could not save game
disk full
```

An error can be *specialized*, giving you the equivalent of an error class hierarchy, testable with `isa`. Do this by calling `.err` on the base error type. (Note the difference between this and *wrapping* an error as we did above.)

```
NetErr = err("network trouble")
e = NetErr.err("host unreachable")
print e isa NetErr
print e isa error
```

```
1
1
```

Errors also carry `.stack`, a stack trace captured where the error was made. Use this to determine where in your program the problem occurred, even if you actually catch and print the error much later.

Errors propagate rather than vanish: if `e` is `error`, then `e + 5` is `e`, `e[i]` is `e`, and most intrinsics hand `e` straight back instead of running.

The `or` operator is different; `e or x` evaluates to `x`. This is a convenient way of handling an error by providing a fallback value.

> **IMPORTANT**
>
> An error that could never be caught halts the program. A bare expression statement that evaluates to an error, `if e then`, `while e`, and reading any member of `e` other than `message`, `inner`, `stack` or `__isa` all terminate with a runtime error. So `file.readLines(path)` on its own line, in case of an error, stops your program on that line; you need to assign the error to a variable or otherwise catch it if you want your program to continue.

Frozen (immutable) maps and lists

Lists and maps in MiniScript have alwasy been *mutable*, meaning that you can change their contents. (Compare this to strings, which are *immutable*; you can't change an existing string, but can only create a new string that may differ from the old one). MiniScript 2 introduces the notion of a **frozen** list or map, which (like a string) is immutable.

There are three basic ways you may get a frozen list or map:

- Some intrinsic methods may return a frozen list or map, e.g. `shellArgs` or `stackTrace`
- Using a list or map as a map key automatically uses a frozen copy
- You may call `freeze` or `frozenCopy` on a list or map yourself.

When a list or map is frozen, any attempt to mutate it generates an error.

```
config = {"host": "localhost", "port": 8080}
freeze config
if isFrozen(config) then print "Yep, it's frozen!"
config.port = 9090
```

```
Yep, it's frozen!
Runtime Error: Attempt to modify a frozen map [line 4]
```

`freeze x` recursively marks `x` and its contents immutable; `isFrozen(x)` tests it; `frozenCopy(x)` returns a frozen copy (or `x` itself, if already frozen).

This also settles the old question of mutable map keys: a list or map used as a map key is automatically stored as a `frozenCopy`, so it cannot change out from under the map. Freeze your keys yourself to skip the copy.

> **TIP**
>
> The core type maps (`list` , `string` , `map` , `number` , `funcRef`) are deliberately *not* frozen — extending them is still normal and expected.

Lists, maps, and computed default parameter values

MiniScript 1.6.2 would not allow you to use a list or map as a default value for a function parameter, nor would it allow a computed expression like `6*7`. MiniScript 2 allows both, as long as the result can be fully evaluated at compile time. Maps and lists used as default values are automatically frozen, ensuring that they can't be corrupted from call to call.

```
f = function(data=[1,2,3], repeats=6*7)
    print data * repeats
end function
```

Property setters

One of MiniScript's design principles is to **hide the difference between storage and computation** from users of a class. That's why you can call a function with no arguments without parentheses; it looks exactly like ordinary stored data, and this is a feature.

But in MiniScript 1, this hiding applied only to *reading* values, never to *writing* them. (Except in cases where a host app made use of `assignOverride`, a feature that no longer exists.)

MiniScript 2 introduces a **property setter** feature: a specially-named method that is invoked when an assignment is made to a map key. The method name is the identifier plus an equal sign, representing assignment.

```
Counter = {"n": 0}
Counter["n="] = function(value)
    if value < 0 then return err("Counter cannot go negative")
    super.n = value        // do the ordinary store
end function

c = new Counter
c.n = 5
print c.n
```

```
5
```

Returning an error from a setter halts the program at the assignment:

```
c.n = -1
```

```
Runtime Error: Uncaught Counter cannot go negative [line 7]
```

Assigning `null` as the setter makes a property read-only:

```
mouse["x="] = null       // mouse.x can now be read but not written
```

> **NOTE**
>
> The bracket form is required (since `x=` is not a legal identifier), and that is deliberate: setters are a library-author feature, and are not intended to be used casually.

`info(x)`

Returns a frozen map describing any value.

```
f = function(x, y=2)
    "Adds two numbers."
    return x + y
end function
print info(@f)
```

```
{"type": "funcRef", "name": "f", "note": "Adds two numbers.", "sourceLoc": "adder.ms
line 1", "params": [{"name": "x", "default": null}, {"name": "y", "default": 2}],
"closure": 1}
```

Every result carries `type`. Beyond that:

- funcRefs report `name`, `note` (the string literal on the first line of the body, like a docstring), `params`, `closure` and `sourceLoc`
- lists report `frozen` and `computed`
- maps report `frozen`
- errors report `message`, `inner`, `stack` and `isa`.

`function` is an expression

`function` ... `end function` is now an expression rather than a statement form, so it can appear anywhere a value can:

```
apply = function(f, x)
    return f(x)
end function

print apply(function(n); return n * n; end function, 7)
```

```
49
```

Dot syntax on numeric literals

```
number.double = function
    return self * 2
end function
print 4.double
print 3.14.double
```

```
8
6.28
```

1.x allowed extension methods on `number` but refused to call them on a literal. 2.0 allows it.

Maps iterate in insertion order

The docs always said map order was undefined. In practice the C# build of 1.x gave insertion order and the C++ build did not always agree. 2.0 *commits* to insertion order, on both builds, including after removals — a re-added key goes to the end.

```
m = {"b": 1, "a": 2, "c": 3}
m.remove "a"
m.d = 4
for kv in m; print kv.key; end for
```

```
b
c
d
```

`range` and `[x] * n` are now cheap

Lists built by `range` or by repeating a single-element list are stored in computed form — three numbers, not a million elements — and materialize only if you mutate them. This is invisible (though fast) except through `info`:

```
print info(range(1, 1000000))
```

```
{"type": "list", "computed": 1, "frozen": 0}
```

So `for i in range(1, 1000000)` no longer allocates a huge list, and is now a performant way to iterate over even a large range of numbers.

`rnd` and `shuffle` are deterministic across platforms

Both now use **xoshiro256+**, so the same seed gives the same sequence on every build and every platform. In 1.x this was platform-dependent.

Mark-and-sweep garbage collector on all platforms

MiniScript 1.x used C#'s standard garbage collector in C#, but a reference-counting system in C++, which could result in leaked memory in the case of reference cycles. In MiniScript 2, a custom mark-and-sweep garbage collector (exposed through the `gc` intrinsic map) is used on all platforms, guaranteeing that even reference cycles are eventually disposed of.

New in command-line MiniScript

- **`key`** — `key.get`, `key.available` and `key.raw`, for character-at-a-time input. Raw terminal mode is entered and restored automatically. Together with the bundled `vt` module, this makes interactive console programs possible.
- **`gc`** — `gc.collect` and `gc.stats`, for inspecting and prodding the garbage collector.
- **REPL conveniences** — `_in` and `_out` (the input and implicit-output history lists) and `reset` (clear all globals and history). The REPL itself gains line editing, input history and syntax coloring.

What hasn't changed

Everything else, as far as we can make it so: the syntax, the standard intrinsics, the `lib` modules, `self` / `super` / `outer` / `globals` / `locals`, chained comparisons, the unary-minus rule for call statements, and the `isa`-based object model. No intrinsic from 1.6.2 was removed.

If you embed MiniScript in a C# or C++ host, see CPP_HOST_UPDATE_GUIDE.md for the API changes; the host-side surface changed considerably more than the language did. Most notably, `assignOverride` is gone; code using it will need to be updated to use setter methods instead.